Print / Save as PDF

BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems
Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an* `__init__` *method to store history.*
   Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`
3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A mathematically perfect Table-Driven Agent is impossible for complex environments because it must store an action for every possible percept sequence. This causes combinatorial explosion. If the number of possible percepts is |P| and the agent operates for T steps, the table may contain |P|^T entries. As the agent's lifetime increases, the percept-sequence history becomes exponentially larger. In Chess, the huge number of legal states and possible game sequences makes such a table impossible to construct or store.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The Condition-Action Rules are implemented by the if statements inside sense_and_act(). The rule if percept.get("food_here"): return "Suck" means IF food is present THEN collect it. The rule if percept.get("wall_ahead"): return "TurnLeft" means IF a wall is detected THEN turn left. The final return "MoveForward" is the default rule used when the other conditions are false. These rules map the current percept directly to an action without using previous percepts or memory.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The Simple Reflex Agent entered a loop because the environment was partially observable and the agent had no percept history. Its sensors provided only local information such as wall_ahead and food_here, while hiding its exact position and previously visited cells. As a result, the same local percept repeatedly triggered the same condition-action rule. For example, every time wall_ahead was true, the agent turned left, even if it had already encountered the same situation. Since it could not remember previous positions, actions, or percepts, it could not recognize that it was repeating a cycle or choose an alternative route.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

My ModelBasedAgent maintains an internal state using relative_position, facing, visited_cells, and last_action. The transition/action model is implemented in _update_internal_state(). If the previous action was MoveForward and the percept confirms that the movement succeeded, the agent updates its relative position and adds the new cell to visited_cells. If the previous action was TurnLeft or TurnRight, it updates its internal facing direction.

The sensor model is represented by percept values such as last_move_succeeded and wall_ahead. These sensor readings tell the agent whether its previous action actually changed the environment. The agent uses this information to avoid incorrectly updating its internal position after a blocked movement. After updating the internal state, it checks whether the cells ahead, left, or right have already been visited. It then applies memory-based IF–THEN rules, such as turning right when a wall is ahead and the left side has already been visited. This allows it to recognize repeated paths and choose an alternative route.

Finalize and Lock Lab 02 Documentation